

Balatro

The first observation to make is that the product of all multiply cards is at most 10^9 . So, we may assume that the number of multiply cards is at most 29, and that $k \leq 29$ (note that k may be much larger than this in the input, but we can just replace it with $\min(k, 29)$).

We will construct an array s where $s[i][j]$ is the highest score achievable using any i mult and j add cards. Once we have this array, for each subsequence length $\ell = 1, \dots, n$ we can just output

$$\max_{1 \leq m \leq \ell} \max_{\substack{i+j=m \\ 0 \leq i \leq k \\ 0 \leq j \leq n}} s[i][j].$$

This can be done in $O(nk)$ time assuming s has already been constructed.

So, how can we construct this array efficiently? We iterate through the cards in order, and each time we encounter a mult card, we process it along with all of the add cards that have not yet been processed (i.e., all add cards between this mult card and the previous mult card, or the beginning of the sequence if this is the first mult card). Note that we will do this at most 29 times. Say that the mult card we are processing has value m and the add cards have values $a_0, a_1, \dots, a_r$.

We first update s to account for the new add cards $a_0, a_1, \dots, a_r$. If we are choosing q of these cards, we will always want to choose the q highest value cards. So, sort the cards in non-increasing order of value and let $b_i = \sum_{0 \leq j \leq i} a_j$. Then, b_q is the best score we can get by picking q of these $r + 1$ add cards. Now, to update s , we do the following for all j and for all i from 1 up to the number of mult cards we have already processed:

$$s[i][j] = \max_{\substack{x+y=j \\ 0 \leq x \leq i \\ 0 \leq y \leq r}} s[i][x] + b_y \tag{1}$$

Doing this naively requires $O(n^2)$ time for each i , but since the b array is concave, we can do better. For fixed i , Eq. (1) is an instance of a $(\max, +)$ -convolution problem with one concave array, so we can solve it in $O(n \log n)$ using a Li Chao tree, or $O(n)$ using SMAWK (the time limit on the problem is set so that either of these approaches should get AC). There are good resources online describing how this can be done¹, so we omit the details here.

Now, we update s to account for the mult card m . Since we are only adding one additional card into consideration now, this is not hard. For all j and for all i from the number of mult cards processed so far down to 1, we do the following:

$$s[i][j] = \max(s[i][j], s[i-1][j] \cdot m)$$

¹<https://codeforces.com/blog/entry/98663>

This takes $O(29 \cdot n)$ time. So, the overall running time is $O(29 \cdot n \log n)$ or $O(29 \cdot n)$ depending on which approach is used for the $(\max, +)$ -convolution.

Bikes and Barricades

This one is simple math. We want to see if any of the barricades cross the positive Y -axis. That's just seeing if the Y -intercept is positive.

First of all, there's no way the barricade could get in Scott's way unless the x values are on opposite sides of the Y -axis. So, we can only consider cases where $x_1 \times x_2 < 0$.

Now then, the general equation of a line is

$$y = mx + b$$

Where m is the slope and b is the Y -intercept – but the Y -intercept is what we're after! So,

$$b = y - mx$$

$$m = \frac{y_2 - y_1}{x_2 - x_1}$$

(Note that since we're only dealing with cases where $x_1 \times x_2 < 0$, there's no danger of $x_2 - x_1 = 0$.)

Then just plug in any point:

$$b = y_1 - mx_1$$

Look to see if b is positive and the smallest you've seen, and that's it.

Here's some Java code for you:

```
int n = sc.nextInt();
double closest = -1.0;
while( n-->0 )
{
    int x1 = sc.nextInt();
    int y1 = sc.nextInt();
    int x2 = sc.nextInt();
    int y2 = sc.nextInt();
    if( x1*x2<0 )
    {
        double m = (double) (y2-y1) / (double) (x2-x1);
        double b = y1 - m*x1;
        if( b>0.0 && (closest<0.0 || b<closest) ) closest = b;
    }
}
ps.println( closest );
```

Call for Problems

This is the simplest problem in the set. Just read a list of integers and count the number of them that are not divisible by 2.

Here's some Java code for you:

```
int n = sc.nextInt();
int count = 0;
while( n-->0 )
{
    if( sc.nextInt()%2==1 ) ++count;
}
ps.println( count );
```

Colorful Trees

First, we can make the tree rooted for convenience.

Then, given an edge between a vertex and its parent like (v, p) , we count the number of same-color pairs that use this edge for their connectivity.

Let's say $\text{cnt}[c]$ is the total number of vertices with color c , and $\text{cnt}[v][c]$ is the number of vertices in the subtree of v (all direct and indirect descendants of v , inclusive).

Clearly, the answer for the edge (v, p) will be:

$\text{Answer}_{(v, p)} = \text{Sum}(\text{cnt}[v][c] * (\text{cnt}[c] - \text{cnt}[v][c]))$ for all possible colors

We observe that in cases where $\text{cnt}[v][c] == 0$ or $\text{cnt}[v][c] == \text{cnt}[c]$, this term is equal to 0, so we can ignore these cases.

We can build $\text{cnt}[v]$ by merging $\text{cnt}[u]$ for all children u of v . These merges can be done using a DSU trick that results in $O(n \log n)$ operations on our collections. We can use either hash maps or tree maps for this.

Consider the $\text{cnt}[v]$ for an imaginary vertex. As we are adding new vertices with any colors to this set, we can maintain the value of:

$\text{Sum}(\text{cnt}[v][c] * (\text{cnt}[c] - \text{cnt}[v][c]))$ for all possible colors

in a variable like $f[v]$. In this way, we won't need to iterate over all existing colors in the set to calculate the final value for that vertex; instead, we can just use the value of $f[v]$.

Here is a sample C++ code:

```
void merge(int v, int u) { // merge u into v
    if (cnv[v].size() < cnv[u].size()) {
        cnv[v].swap(cnv[u]);
        swap(f[v], f[u]);
    }

    for (auto [x, cnx] : cnv[u]) {
        int av = cnv[v][x] * (cnt[x] - cnv[v][x]);
        f[v] -= av;

        cnv[v][x] += cnx;

        av = cnv[v][x] * (cnt[x] - cnv[v][x]);
        f[v] += av;

        if (av == 0) { // Not necessary
            cnv[v].erase(x);
        }
    }
}

void dfs(int v, int par = -1) {
    cnv[v][c[v]] = 1;
    f[v] = cnt[c[v]] - 1;

    for (auto [u, i] : g[v]) {
        if (u != par) {
            dfs(u, v);
            ans[i] = f[u];
            merge(v, u);
        }
    }
}
```

Dishonest Lottery

There are only 50 possible drawn numbers – so just create an array of counts, and print out any with more than $2 \times n$. The toughest part is the output formatting, but the Kattis judge is very forgiving with whitespace.

Here's some Java code for you:

```
int counts[] = new int[51];
Arrays.fill( counts, 0 );

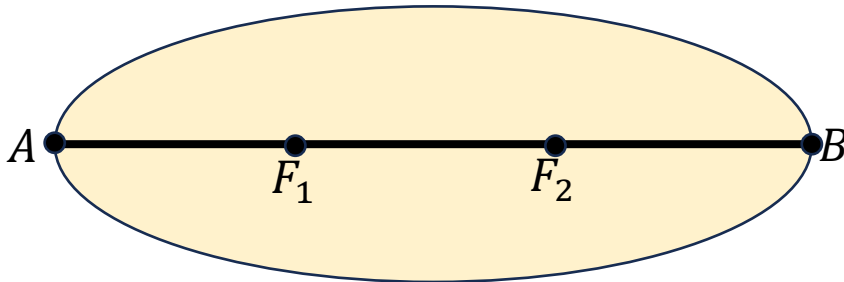
int n = sc.nextInt();
for( int i=0; i<10*n; i++ )
{
    for( int j=0; j<5; j++ )
    {
        ++counts[sc.nextInt()];
    }
}

boolean atLeastOne = false;
for( int i=1; i<=50; i++ )
{
    if( counts[i]>n+n )
    {
        ps.print( (atLeastOne?" ":"") + i );
        atLeastOne = true;
    }
}

ps.println( atLeastOne ? "" : "-1" );
```

Ellipse Eclipse

An Ellipse is the set of all points where the sum of the distances to two foci is constant. You are given the coordinates of the two foci and the length of the Major Axis. What is that constant sum of distances. It turns out, it's just the length of the Major Axis! Look at this:



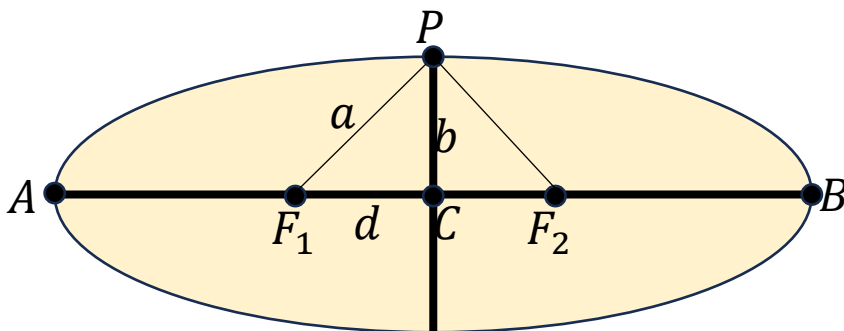
That constant is the sum of AF_1 and AF_2 . But, since ellipses are symmetrical, AF_1 is the same as BF_2 ! Then $AF_2 + BF_2$ is just the Major Axis.

OK, now that we've got that, most programmers will probably use some combination of binary/ternary search to nail down the max/mins, but this can be done closed form, without searching, using a bit of trigonometry and calculus.

Consider an ellipse with its center at the origin and its Major Axis along the X -axis. I know that's not the case, but we'll start there. Such an ellipse can be represented parametrically:

$$x = a \sin \theta, y = b \cos \theta$$

Where a is the length of the Semimajor Axis (half of the Major Axis) and b is the length of the Semiminor Axis (which is the smallest radius, perpendicular to the Major Axis). The first, a , is just half of our Major Axis input. Now, we've got to find b .



The Semiminor is CP . We know that F_1PF_2 is the length of the Major Axis (please excuse my poor drawing), so F_1P is the length of the Semimajor. We know that the center C is exactly halfway between F_1 and F_2 . So, $C = (\frac{x_1+x_2}{2}, \frac{y_1+y_2}{2})$. (Yeah, we could just take $\frac{F_1F_2}{2}$, but we'll need C later).

Now, let $d = F_1C$, and we can use the Pythagorean Theorem to get $b = \sqrt{a^2 - d^2}$

OK, that would be great if the ellipse were centered at the origin with its Major Axis along the X-axis, but that's not the case. Let's deal with the angle first. We can use `atan2` to get the angle of the Major Axis:

$$\alpha = \text{atan2}(y_2 - y_1, x_2 - x_1)$$

OK, everything is rotated by α . To rotate, the new coordinates are:

$$x' = x \cos \alpha - y \sin \alpha$$

$$y' = x \sin \alpha + y \cos \alpha$$

We have the parametric representation of the points of the ellipse, plug'em in!

$$x = a \cos \theta \cos \alpha - b \sin \theta \sin \alpha$$

$$y = a \cos \theta \sin \alpha + b \sin \theta \cos \alpha$$

Now, we want to find the max/min of both x and y . Those are inflection points – so all we have to do is take the derivative of each, set it to 0 and solve for θ (since that's the only variable here). We'll do x and let you do y , remembering that the derivative of $\sin \theta$ is $\cos \theta$ and the derivative of $\cos \theta$ is $-\sin \theta$.

$$\frac{d}{d\theta} a \cos \theta \cos \alpha - b \sin \theta \sin \alpha = -a \sin \theta \cos \alpha - b \cos \theta \sin \alpha = 0$$

$$a \sin \theta \cos \alpha = -b \cos \theta \sin \alpha$$

$$\frac{\sin \theta}{\cos \theta} = \frac{-b \sin \alpha}{a \cos \alpha}$$

$$\theta = \text{atan2}(-b \sin \alpha, a \cos \alpha)$$

Now, we can just plug that value of θ back into $x = a \cos \theta \cos \alpha - b \sin \theta \sin \alpha$ to get a max or min value of x . but wait! We don't know if that's a max or a min, and we're still assuming that the center is at the origin! That's OK. Take the absolute value, and that's a distance from the center in the X direction.

So, let $dx = |a \cos \theta \cos \alpha - b \sin \theta \sin \alpha|$, compute dy similarly, and then our bounding box goes from $(C_x - dx, C_y - dy)$ to $(C_x + dx, C_y + dy)$. I told you we'd need that center!

Here's some Java code for you:

```
a = a/2.0;
double cx = (x1+x2)/2.0;
double cy = (y1+y2)/2.0;

double dx = cx-x1;
double dy = cy-y1;
double d = Math.sqrt( dx*dx + dy*dy );
double b = Math.sqrt( a*a - d*d );

double alpha = -Math.atan2( y2-y1, x2-x1 );
double cosa = Math.cos( alpha );
double sina = Math.sin( alpha );

double theta = Math.atan2( -b*sina, a*cosa );
dx = Math.abs( a*Math.cos(theta)*cosa - b*Math.sin(theta)*sina );
double xlo = cx - dx;
double xhi = cx + dx;

theta = Math.atan2( b*cosa, a*sina );
dy = Math.abs( a*Math.cos(theta)*sina + b*Math.sin(theta)*cosa );
double ylo = cy - dy;
double yhi = cy + dy;

DecimalFormat df = new DecimalFormat( "0.000000" );
ps.println( df.format( xlo ) + " " + df.format( ylo )
    + " " + df.format( xhi ) + " " + df.format( yhi ) );
```

Gears and Axles

How do you maximize the rate of spin? Mesh big gears with little ones. So, for every tooth size, sort them and match the biggest with the smallest, the next biggest with the next smallest, and so on.

Suppose you have 6 gears with the same size teeth. In sorted order, they are A, B, C, D, E, F .

The best rate of spin you can get is $\frac{A}{F} \times \frac{B}{E} \times \frac{C}{D}$.

If you take the natural log, that becomes $\ln A - \ln F + \ln B - \ln E + \ln C - \ln D$.

You start at 1hz. $\ln 1 = 0$. So, all you have to do is keep a running sum of the above for all the teeth, and the result is your answer.

Here is some Java code for you:

```
ArrayList<Double> gears[] = new ArrayList[100000];
Arrays.fill( gears, null );

int n = sc.nextInt();
while( n-->0 )
{
    int s = sc.nextInt()-1;
    int c = sc.nextInt();
    if( gears[s]==null ) gears[s] = new ArrayList<Double>();
    gears[s].add( Math.log( c ) );
}

double angular = 0.0;
for(ArrayList<Double> gear : gears ) if( gear!=null )
{
    n = gear.size();
    Collections.sort( gear );
    for( int i=0; i<n/2; i++ )
    {
        angular += gear.get( n-i-1 ) - gear.get( i );
    }
}
ps.println( angular );
```

Genetic Reconstruction

For each creature, we already know one of its alleles, which corresponds to its eye color. This allele is the alphabetically first one. Additionally, the other allele must be greater than or equal to the eye color (because the eye color is the minimum of the two alleles).

For each creature, we must decide which parent passes down the eye color and which one passes down the other allele. This gives us 2^n possible configurations since we need to make this choice for each creature.

Assume we know which parent gives the eye color and which gives the other allele for each creature. For simplicity, let's say the eye color comes from p1 (parent 1), and the other allele comes from p2 (parent 2).

Define $c[i]$ as the eye color (the first allele) of creature i and $g[i]$ as the second allele. We will iterate over all creatures to assign values for $g[i]$, initializing them to -1 to indicate they are not yet set.

Now, consider the following:

1. If $c[i]$ matches either $c[p1[i]]$ or $g[p1[i]]$ (alleles from parent 1), there are no additional constraints.
2. Otherwise, we need to set $g[p1[i]]$ to $c[i]$. If $g[p1[i]]$ is already set to a different value, this configuration is invalid, and we skip it.

Next, if $g[i]$ is already set:

1. If $g[i]$ matches either $c[p2[i]]$ or $g[p2[i]]$ (alleles from parent 2), there are no additional constraints.
2. Otherwise, we set $g[p2[i]]$ to $g[i]$. If $g[p2[i]]$ is already set to a different value, this configuration is invalid, and we skip it.

If $g[i]$ is not yet set, it must satisfy a minimum requirement: it must be at least $c[i]$. This means we need to ensure that one of the alleles from $p2[i]$ is at least $c[i]$. We update this minimum constraint for $p2[i]$ as $h[p2[i]] = \max(h[p2[i]], h[i])$.

Since creatures place constraints on their parents, we should process them in reverse order (from creature n down to 1).

In a second pass, we iterate through the creatures in increasing order (from 1 to n), now that parent information is fixed. We assign the smallest possible value for $g[i]$ that is compatible with their parent $p2$.

Here is a sample c++ code:

```
string ans = "z";

void check(int msk) {
    memset(g, -1, sizeof g);
    memset(h, 0, sizeof h);
    bool bad = false;

    // Flip parents for creatures based on the bitmask.
    for (int i = 0; i < n; i++) {
        if ((1 << i) & msk) {
            swap(p1[i], p2[i]);
        }
    }

    // Reverse order to propagate constraints from creatures to parents.
    for (int i = n - 1; i >= 0; i--) {
        h[i] = max(h[i], c[i]);

        // Skip if no parents.
        if (p1[i] == -1) continue;

        // First allele constraint: c[i] comes from p1.
        if (c[p1[i]] != c[i]) {
            if (g[p1[i]] != -1 && g[p1[i]] != c[i]) {
                bad = true;
                break;
            }
            g[p1[i]] = c[i];
        }

        // Second allele constraint: g[i] comes from p2.
        if (g[i] != -1) {
            if (c[p2[i]] != g[i]) {
                if (g[p2[i]] != -1 && g[p2[i]] != g[i]) {
                    bad = true;
                    break;
                }
                g[p2[i]] = g[i];
            }
        }
        else if (h[i]) {
            h[p2[i]] = max(h[p2[i]], h[i]);
        }
    }

    // Now assign values for g[i] in increasing order.
    string tmp;
    for (int i = 0; i < n; i++) {
        if (p2[i] == -1) {
            if (g[i] == -1) {
                g[i] = h[i];
            }
        }
        else {
            if (g[i] == -1) {
```

```

        if (c[p2[i]] >= h[i]) {
            g[i] = c[p2[i]];
        } else {
            g[i] = g[p2[i]];
        }
    }
}

// Check if the assignment is invalid.
if (g[i] < h[i]) {
    bad = true;
    break;
}

// Build the solution string.
tmp += char('a' + c[i]);
tmp += char('a' + g[i]);
tmp += '\n';
}

// Revert parent swaps after checking.
for (int i = 0; i < n; i++) {
    if ((1 << i) & msk) {
        swap(p1[i], p2[i]);
    }
}

// Update answer if valid.
if (!bad) {
    ans = min(ans, tmp);
}
}

// Iterate through all possible configurations.
for (int msk = 0; msk < (1 << n); msk++) {
    check(msk);
}

```

Light Up

There are three ways a solution can fail:

- If a light bulb shines on another light bulb
- If a blocked cell with a number has the wrong number of light bulbs around it
- If an open cell doesn't get lit

So, this problem is just some 2D array manipulation.

- For every cell:
 - If it's a light bulb ('?'), keep going up, down, left, right, marking open cells
 - If you see another light bulb, FAIL
 - If it's a number ('0', '1', '2', '3', '4') look 1 cell up, down, left, right, count the light bulbs
 - If the count doesn't match the number, FAIL

After that, go through the grid again.

- For every cell:
 - If it's an unmarked open cell, FAIL

Here's some Java code:

```
boolean ok = true;
for( int i=0; i<n; i++ ) for( int j=0; j<n; j++ )
{
    if( grid[i][j]=='?' )
    {
        int d;
        for( d=i-1; d>=0 && (grid[d][j]=='.' || grid[d][j]=='#'); --d ) grid[d][j] = '#';
        if( d>=0 && grid[d][j]=='?' ) ok = false;
        for( d=i+1; d<n && (grid[d][j]=='.' || grid[d][j]=='#'); ++d ) grid[d][j] = '#';
        if( d<n && grid[d][j]=='?' ) ok = false;
        for( d=j-1; d>=0 && (grid[i][d]=='.' || grid[i][d]=='#'); --d ) grid[i][d] = '#';
        if( d>=0 && grid[i][d]=='?' ) ok = false;
        for( d=j+1; d<n && (grid[i][d]=='.' || grid[i][d]=='#'); ++d ) grid[i][d] = '#';
        if( d<n && grid[i][d]=='?' ) ok = false;
    }
    else if( Character.isDigit( grid[i][j] ) )
    {
        int constraint = grid[i][j] - '0';
        int count = 0;
        if( i-1>=0 && grid[i-1][j]=='?' ) ++count;
        if( i+1<n && grid[i+1][j]=='?' ) ++count;
        if( j-1>=0 && grid[i][j-1]=='?' ) ++count;
        if( j+1<n && grid[i][j+1]=='?' ) ++count;
        if( count!=constraint ) ok = false;
    }
}
for( int i=0; i<n; i++ ) for( int j=0; j<n; j++ ) if( grid[i][j]=='.' ) ok = false;
ps.println( ok ? 1 : 0 );
```

Menger Sponge

Like all fractals, the Menger sponge exhibits self-similar structure, and so this problem is inviting a recursive solution. Let's consider the base case first, and then build a full solution. We will store the coordinates as an exact rational number (using the `Rational` package in Python, for instance, or a custom class in C++) in array `coords[3]`.

Base Case: $L = 1$

By inspecting the $L = 1$ cube, we notice a pattern in the subcubes that have been deleted: all cubes that are in *at least two* of the middle row, middle column, or middle slice of the $3 \times 3 \times 3$ grid of subcubes are deleted. We can turn this insight into a predicate that evaluates whether a point is inside the $L = 1$ cube:

```
function INCUBE(coords)
    count = 0
    for  $i = 0 \dots 2$  do
        if  $\frac{1}{3} < \text{coords}[i] < \frac{2}{3}$  then
            count = count + 1
        end if
    end for
    return count < 2
end function
```

Note the use of *strict* inequality, to correctly implement the requirement in the problem statement that points exactly on the boundary of cubes count as being in the cube.

Recursive Case

For a cube with $L > 1$, we can first check if the query point is in the level-1 cube. If not, the point is definitely not in the level L cube. Notice that each of the 20 different $\frac{1}{3} \times \frac{1}{3} \times \frac{1}{3}$ subcubes of the level L cube are themselves Menger sponges of level $L - 1$, shrunk by a factor of 3 and translated from the origin to the appropriate spot in the $3 \times 3 \times 3$ grid of subcubes.

We can undo this translation and shrinking to recursively query whether a point in the level-1 cube is in the level- L cube:

```
function MENER(L, coords)
    if  $L == 0$  then
        return true
    end if
    if not inCube(coords) then
        return false
    end if
    for  $i = 0 \dots 2$  do
        if  $\text{coords}[i] < \frac{1}{3}$  then
            shift[i] = 0
        else if  $\text{coords}[i] < \frac{2}{3}$  then
            shift[i] =  $\frac{1}{3}$ 
        else
            shift[i] =  $\frac{2}{3}$ 
        end if
        coords[i] =  $3 \cdot (\text{coords}[i] - \text{shift}[i])$ 
    end for
    return menger(L - 1, coords)
end function
```


This solution has time complexity $O(L)$. The problem statement guarantees that $L \leq 10^5$, and so the above recursive solution should fit within the stack space of most programming languages. It's also straightforward to turn the above recursive code into an iterative algorithm, if necessary.

Alternate Approach: Ternary Expansion

Notice a couple of facts about the ternary decimal [sic] representation of `coords[i]` which are particularly convenient for solving this problem:

- The condition $\frac{1}{3} < \text{coords}[i] < \frac{2}{3}$ is equivalent to $0.1_3 < \text{coords}[i] < 0.2_3$;
- Shifting `coords[i]` and multiplying by three is the same as a *left shift* operation on `coords[i]` in ternary.

To make it easier to iterate over them, let us store the numerators and denominators in arrays `num[3]` and `denom[3]` (so that `num[0] = xnum` etc.). We can then use the above insights to lazily expand `coords[i]` in ternary one digit at a time to see if the query point is in the level-0, ..., L cube:

```

for  $i = 0 \dots L$  do
  count = 0
  for  $j = 0 \dots 2$  do
    quot = num[j] / denom[j]
    rem = num[j] % denom[j]
    if quot == 1 && rem  $\neq$  0 then                                 $\triangleright$  The second condition enforces strict inequality
      count = count + 1
    end if
    num[j] = 3 * rem                                               $\triangleright$  Left-shift the ternary decimal
  end for
  if count  $\geq$  2 then
    return false
  end if
end for
return true

```

This solution also runs in time $O(L)$, but has the advantage of not needing any kind of exact rational number class.

Rhythm Flow

At first blush this problem looks like it's a vanilla assignment problem (or maximum-weight bipartite matching). But a naive implementation using e.g. the Hungarian algorithm won't pass the samples: a maximum-weight matching might violate the constraint that if two actual button presses are matched to expected button presses, the earlier actual press must match the earlier expected press.

But this same restriction allows us to formulate a problem solution using dynamic programming. Let $dp[i][j]$ be the maximum possible score after matching the first i actual presses to the first j expected presses. We can fill in this DP table row-by-row:

```
for j = 0...n do
  dp[0][j] = 0
end for
for i = 1...m do
  for j = 0...n do
    dp[i][j] = dp[i-1][j]           ▷ Do not match actual press i to any expected press
  for k = j; k ≥ 1; k = k - 1 do
    score = s(ai, ek) + dp[i-1][k-1]           ▷ Match actual press i to expected press k
    dp[i][j] = max(dp[i][j], score)
  end for
end for
end for
```

where $s(a, e)$ is the score listed in the table. The answer to the problem is then $dp[m][n]$.

Improving the Time Complexity

The above algorithm runs in $O(n^2m)$ time, which is too slow. But notice that the above DP can be improved to remove the inner loop completely. Instead of searching over all possible matches for actual press i , we can reuse $dp[i][j-1]$, which already computes the best possible score when matching all actual presses up to press i with all expected presses $0 \dots j-1$:

```
for j = 0...n do
  dp[0][j] = 0
end for
for i = 1...m do
  dp[i][0] = 0
  for j = 1...n do
    score1 = s(ai, ej) + dp[i-1][j-1]           ▷ Match actual press i to expected press j
    score2 = dp[i][j-1]                             ▷ Match actual press i to some expected press < j
    score3 = dp[i-1][j]                             ▷ Don't match actual press i at all
    dp[i][j] = max(score1, score2, score3)
  end for
end for
```

This solution runs in time $O(nm)$. (Note also that it's not necessarily to keep the entire DP table in memory: you only need the current and previous rows, though this optimization is not needed to fit within the problem memory limit.)

Alternate Solution

Instead of removing the inner loop of the $O(n^2m)$ solution, it's also possible to optimize it by noticing that it never makes sense to match actual button press i to expected button press k once the distance between a_i and e_k is greater than 102 milliseconds:

```
for j = 0...n do
```

```

    dp[0][j] = 0
end for
for i = 1 ... m do
    for j = 0 ... n do
        dp[i][j] = dp[i-1][j]                                ▷ Do not match actual press i to any expected press
        for k = j; k ≥ 1; k = k - 1 do
            if ai - ek > 102 then
                break
            end if
            score = s(ai, ek) + dp[i-1][k-1]                ▷ Match actual press i to expected press k
            dp[i][j] = max(dp[i][j], score)
        end for
    end for
end for
end for

```

The time complexity is now $O(nm)$ (with a worst-case $102\times$ constant factor due to the inner loop), which is enough to solve the problem within the time limit.

Snake

For the parameters given, the problem can be solved by exploring all possible positions that are reachable from the initial position of the snake.

Each position can be encoded using $L+1$ parameters by specifying the position of the head, the relative position of the neck (the second snake piece following the head), and then a series of left, right or straight turns. For instance, the following position:

	0	1	2	3	4	5
0						
1		0				
2		1	2	3		
3				4		
4				5	6	
5						

Could be encoded as

((1, 1), Down, Left, Straight, Right, Straight, Left)

Therefore, an upper bound on the number of possible states is $4mn3^{L-2}$. ($m \times n$ possible positions for the head, Up/Down/Right/Left for the neck, and then Left/Straight/Right for the remainder of the snake.) This is a theoretical maximum of ~191M states. In addition, we can use the following observation to shorten the search: If the snake ever manages to enter a square previously occupied by its body, it will be able to get to the apple because it has found a circular path on which it can move and then take a turn straight to the apple.

Therefore, this problem can be solved by exploring all possible states using a breadth-first search with a reasonably efficient encoding of the snake's position.

Be aware that the snake cannot run backwards, this input should yield 0 (no).

3 1
0
1
A

